Lab 5: Introduction to Flex

☑ What is Flex?

Flex = **Fast Lexical Analyzer** — an open-source, improved version of Lex. **F**ast **Lex**ical Analyzer) is a tool for **generating scanners** — programs that break input text into **tokens** (words, numbers, symbols).

It reads a .l file (your scanner rules) and generates C code (lex.yy.c).
Then you compile that C code to get a working lexer.

Flex: generates scanners (lexers) from regular grammar rules; latest release is version 2.6.4 (May 2017) under a BSD license Wikipedia +1 GitHub +1 .

Bison: generates parsers (Yacc-compatible) from grammar definitions; latest release is in the Bison 3.x series under GPL with a parsed-code exception Wikipedia .

WinFlexBison: bundles versions of both for Windows users. As of the latest v2.5.25 (released Jan 11, 2022), it includes Bison 3.8.2 and Flex 2.6.4

📄 Basic Structure of a Flex file

A Flex file has **three sections**, separated by %%:

```
%{
  // C code to include (e.g., headers, global variables, function declarations)
%}

%%
  // Patterns and actions
%%

  // C functions (e.g., main)
```

☑ Step 2: Run Flex

```
flex example.l
gcc lex.yy.c -o example
```

./example (example.exe)

☑ Structure of a Lex Program

```
%{
   // C code section: declarations, headers
   #include <stdio.h>
   int count = 0;
%}

%%
   // Lex rules go here
   [0-9]+      { /* actions using if/loop/switch go here */ }
   .|\n        { /* catch-all rule */ }
%%

int main() {
   yylex(); // start scanning
   return 0;
}
```

☑ 1. yytext

yytext is a **global character array** (i.e., char* yytext) that contains the **current token** (text) matched by Lex.

```
[0-9]+   { printf("NUMBER: %s\n", yytext); }
```

- If the input is 123, then yytext contains "123".
- Access the value of the token.
- Convert it (e.g., using atoi(yytext) to get an integer).

☑ 2. yywrap()

yywrap() is a function Lex calls **when it reaches the end of the input file**.

```
int yywrap() {
```

```
    return 1;  // means "no more files"
}
```

Purpose:

- If you want to process more files, return 0 (false).
- If you're done, return 1 (true).

Required when compiling:

If not using %option noyywrap, you **must define** this function, or you'll get a linker error.

☑ 3. yylex()

yylex() is the **main function** generated by Lex. It performs **lexical analysis**—it reads the input and matches tokens.

- Scans input using the rules you defined.
- For each match, it executes the associated action (e.g., a printf()).

Example main():

```
int main() {
    yylex();  // start scanning input
    return 0;
}
```

Each call to yylex() returns the next token (if integrated with Bison). If you're only using Lex, it processes the whole input.

☑ Summary Table

Component	Meaning	Purpose
yytext	Current matched token	Access the matched text

yywrap	End-of-input handler	Return 1 to end scanning
yylex	Main scanner function	Start lexical analysis

Problem 1: Write a lex program to recognize numbers, words and white space characters.

Code:

```
%{
#include<stdio.h>
int count = 0;
%}

%%
[0-9]+            {printf("Numbers: %s\n", yytext);}
[0-9]+\.[0-9]+  {printf(" Float Numbers: %s\n", yytext);}
[A-Za-z]+       {printf("Words: %s\n", yytext);}
[ \t\n]           {printf("White space characters\n");}
.                  ;

%%

int yywrap(){
   return 1;
}

int main()
{
   yylex();
   return 0;

}
```

Some common regular expressions used in lex:

Regular Expression	Description / Matches	Examples
`[a-zA-Z_][a-zA-Z0-9_]*`	Identifier (variable/function names)	`myVar`, `_temp`, `x1`
`[0-9]+`	Integer numbers	`42`, `123`, `0`
`[0-9]+\.[0-9]+`	Floating point numbers (with digits before & after `.`)	`3.14`, `0.99`
`"if"`, `"else"`, `"while"`	Keywords (matched as string literals)	`if`, `else`, `while`
"+"	"-"	"*"
"="	"=="	"!="
[\t\n]+	Whitespace (space, tab, newline)	(usually ignored)
"("	")"	";"
"//".*	Single-line comments (C/C++ style)	`// comment here`

"/*"([^	*+[^/])*+"/"	Multi-line comments (C-style)
.	Any other single character (catch-all for unknowns)	@, #, ^

Conditional Statement:

🔁 1. Using if, else if, else in Lex

```
[A-Za-z] {
   char c = yytext[0];
   if (c == 'a' || c == 'A') {
      printf("Found an 'a' or 'A'\n");
   } else if (c == 'z' || c == 'Z') {
      printf("Found a 'z' or 'Z'\n");
   } else {
      printf("Found other letter: %c\n", c);
   }
}
```

🔁 2. Using for, while, do-while Loops

```
[0-9]+ {
   int sum = 0;
   for (int i = 0; yytext[i] != '\0'; i++) {
      sum += yytext[i] - '0';
   }
   printf("Sum of digits = %d\n", sum);
}
```

You can also use while or do-while in similar fashion.

🔁 3. Using switch-case Statement

```
[A-Za-z] {
   switch (yytext[0]) {
      case 'a': case 'A':
         printf("It's A\n");
         break;
      case 'b': case 'B':
         printf("It's B\n");
         break;
      default:
         printf("Other letter: %c\n", yytext[0]);
   }
}
```

Problem 2: Write a Flex program that reads input text and counts:

- Number of vowels (a, e, i, o, u) — case-insensitive
- Number of consonants (letters other than vowels)

Hint:

- [aeiouAEIOU] → vowels
- [A-Za-z] → letters
- Use if inside the action to separate consonants

Code:

```
%{
#include <stdio.h>
int vowel_count = 0;
int consonant_count = 0;
%}
%%
[A-Za-z] {
```

```
    char c = yytext[0];
    if (c == 'a' || c == 'A' ||
        c == 'e' || c == 'E' ||
        c == 'i' || c == 'I' ||
        c == 'o' || c == 'O' ||
        c == 'u' || c == 'U') {
        vowel_count++;
    } else {
        consonant_count++;
    }
}
.|\n   ; // Ignore everything else
%%
int yywrap() { return 1; }
int main() {
    yylex();
    printf("Total vowels: %d\n", vowel_count);
    printf("Total consonants: %d\n", consonant_count);
    return 0;
}
```

Problem2: Write a Flex program that counts:

Lab 5: Introduction to Flex

- +
- -
- *
- /

Hint:

"+" { plus_count++; }

"-" { minus_count++; }

"*" { star_count++; }

"/" { slash_count++; }

Code:

```
%{
 #include <stdio.h>
int plus_count = 0;
int minus_count = 0;
int star_count = 0;
int slash_count = 0;
%}
%%
"+" { plus_count++; }
"-" { minus_count++; }
"*" { star_count++; }
```

```
"/" { slash_count++; }

.|\n ; // Ignore other characters and newlines %%

int yywrap() { return 1; }

int main() {

yylex();

printf("Total '+': %d\n", plus_count);

printf("Total '-': %d\n", minus_count);

printf("Total '*': %d\n", star_count);

printf("Total '/': %d\n", slash_count);

return 0;

}
```

Problem3: Identify keywords vs identifiers
Input:

- If token is a keyword (int, if, while) → print KEYWORD: <token>
- Else, if it's an identifier → print IDENTIFIER: <token>

Hint:

```
"int"|"if"|"while"  { printf("KEYWORD: %s\n", yytext); }

[A-Za-z_][A-Za-z0-9_]*  { printf("IDENTIFIER: %s\n", yytext); }
```

Code:

```
%{

#include <stdio.h>
```

Lab 5: Introduction to Flex

```
%}

%%

"int"     { printf("KEYWORD: %s\n", yytext); }

"if"      { printf("KEYWORD: %s\n", yytext); }

"while"   { printf("KEYWORD: %s\n", yytext); }

[A-Za-z_][A-Za-z0-9_]*   { printf("IDENTIFIER: %s\n", yytext); }

[ \t\n]+  ;   // Ignore whitespace

.         ;   // Ignore any other single character

%%

int yywrap() { return 1; }

int main() {

    yylex();

    return 0;

}
```

Problem 5: Design a lexical analyzer.

```
%{
#include <stdio.h>
#include <string.h>

int keyword_flag = 0;

char *keywords[] = {
    "int", "float", "if", "else", "while", "for", "return", "char", "double", "void", "do",
"break", "continue", "switch", "case"
};

int isKeyword(char *word) {
```

Lab 5: Introduction to Flex

```
    for (int i = 0; i < sizeof(keywords)/sizeof(keywords[0]); i++) {
        if (strcmp(word, keywords[i]) == 0)
            return 1;
    }
    return 0;
}
%}

%%

[ \t\n]+                ; // Ignore whitespace

"=="|"!="|"<="|">="     { printf("OPERATOR: %s\n", yytext); }

"+"|"-"|"*"|"/"|"="|"<"|">"  { printf("OPERATOR: %s\n", yytext); }

[0-9]+(\.[0-9]+)?       { printf("NUMBER: %s\n", yytext); }

[a-zA-Z_][a-zA-Z0-9_]*  {
                        if (isKeyword(yytext))
                            printf("KEYWORD: %s\n", yytext);
                        else
                            printf("IDENTIFIER: %s\n", yytext);
                    }

[(){}\[\];,]            { printf("SPECIAL CHARACTER: %s\n", yytext); }

.                   { printf("UNKNOWN: %s\n", yytext); }

%%

int main() {
    yylex();
    return 0;
}

int yywrap() {
```

```
    return 1;
}
```